机器学习笔记　表示不熟　半重点　重点　补充说明

Task0 基础知识

数学基础

偏导数：用来求多变量函数中一个变量的导数.

链式法则：$\frac{\partial f}{\partial \alpha} = \frac{\partial f}{\partial \beta} \cdot \frac{\partial \beta}{\partial \alpha}$ → 求梯度.

梯度：$\frac{\partial L}{\partial W} / \frac{\partial L}{\partial}$

矩阵乘法：matrix1@matrix2

线性变换：Linear

矩阵求导：

矩阵乘法表示神经网络的全连接层前向传播：

编程基础

变量：✓

判断："!="不等于 "=="等于 / if <条件句>: / else if = elif <条件句> / else:　从上到下判断，若有条件符合则✓

循环：for .... in range()//dict/list>: or while <条件>:

函数：进行一些命令操作，不存储参数

类：制造实例(函数+可学习参数)的"模版"

模块：把函数分组，打包成一个文件⇒模块

按目录来组织模块的方法：→包.

顶层包名：

my-package

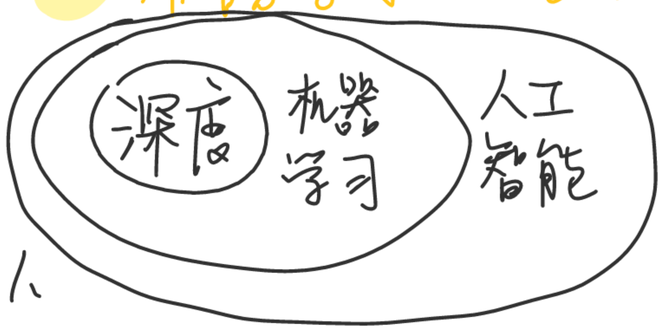
- __init__.py
- abc.py
- xyz.py

≫ 两个模块　引用时写"import my-package.xyz"

安装包："pip install <包名>"

机器学习入门笔记

1.



2. 监督 vs. 无监督

监督 ↓ 标签.正确答案 猫狗分类

无监督 ↓ 无正确答案 单分类不贴标签.

3. 数据、标签. 特征

数据 ↓ 要研究的对象

标签 ↓ 要的答案

特征 ↓ 相关题干信息

Train / Val / Test

Train ↓ 训练

Val ↓ 验证 模拟 选择模型

Test ↓ 最终实战.

4. batch size ⇒ 把数据+标签打包在一起的 batch 的大小.

有什么影响?

5. 模型 ⇒ 一种固定的数学处理. 内含可学习参数

模型参数.

6. 神经网络组成

Input → Linear → ReLU → Linear → Output.

激活函数：非线性函数

1. ReLU(x) = max(0, x)　参数要么 x、要么 0
2. Sigmoid(x) = $\frac{1}{1+\exp(-x)}$ → 把输出压缩 (0, 1)
3. Tanh：类似 Sigmoid. → (-1, 1)

7. 矩阵求导

对于：$x=\begin{bmatrix}x_1\\x_2\end{bmatrix}$, $y=\begin{bmatrix}y_1\\y_2\end{bmatrix}$.

求 x 对 y 的导数

$J_{ij}=\frac{\partial y_i}{\partial x_i}$

$J=\begin{bmatrix}\frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2}\\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2}\end{bmatrix}$

雅可比矩阵.

矩阵对矩阵：$f(X)\in\mathbb{R}^{p\times q}$

$\mathbb{R}^{m\times n}$ ← 把 X、Y 向量化.

变成一个大形的可比矩阵：

$\frac{\partial vec(Y)}{\partial vec(X)}\in R^{pq\times mn}$

长度 pq 的列向量　长度 mn 列向量

一个实数矩阵（pq行 × mn列）的集合.

$T_{kl,ij}=\frac{\partial Y_{kl}}{\partial X_{ij}}$　$k=1,\dots,p$　$l=1,\dots,q$　$i=1,\dots,m$　$j=1,\dots,n$

T 为四阶张量　形状为 p×q×m×n

$X=\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}\Rightarrow vec(X)=\begin{bmatrix}1\\3\\2\\4\end{bmatrix}$ / $vec_{row}(X)=\begin{bmatrix}1\\2\\3\\4\end{bmatrix}$ Numpy reshape 行优先.

8. 用矩阵乘法表示神经网络的全连接层前向传播：

$$X \xrightarrow{W_1, b_1} Z_1 \xrightarrow{ReLU} H \xrightarrow{W_2, b_2} O$$

$$\begin{cases} Z_1 = XW_1^T + b_1 \\ H = ReLU(Z_1) \\ O = HW_2^T + b_2 \end{cases}$$

9. 前向传播、反向传播
   - 前向传播 → 更新参数
   - 反向传播 → 得到下一次更新参数的方向
     - 损失函数算损失、明确方向
     - 梯度：方向
     - 学习率：改进的速率

10. 优化器 → 更新参数

11. 过拟合、背下来了
    欠拟合：没学会
    正则化：防过拟合的方法
    - Dropout
    - 早停
    - L2正则化：引入正则化系数、实现权重衰减（在 optimizer 中）

Python 热身（hello_ml.py）

列表（list）

例：names = ["甲", "乙", "丙", "丁"]

1、 len() 获得元素个数.

字典

```
scores = {
    "甲": 99,
    ⋮
    "丁": 66
}
```

2、
```
def average(scores):
    return sum(scores.values()) / len(scores)
```

3、 使用 numpy 创建矩阵并乘法：

```
import numpy as np
mat1 = np.array([
        [1., 2.],
        [3., 4.],
        [5., 6.]
])
                        result = mat1 @ mat2
print("Shape of mat1:", mat1.shape)
```

4.
```
import torch                    torch.device("cuda")
mat3 = torch.tensor([
        [1., 2.],
        [3., 4.]
])
```

搭建开发环境.

1. 不同项目 → 不同 Python 虚拟环境：固定下来这一个项目所用的配置. 避免全局设置时发生依赖冲突. /更新/改动

2. CPU、GPU 各自擅长什么？
   - 逻辑推理和简单计算
   - 并行巨量计算.

为什么经常使用 GPU？

3. CUDA？ compute unified device architecture. 统一计算设备架构 / NVIDIA 提供的 GPU 编程接口

CUDA. 显卡驱动. PyTorch?

显卡驱动 → 与GPU硬件通信

PyTorch Python API → PyTorch C++ API(ATen) → CUDA Runtime 函数 → 显卡驱动 → 与硬件通信

PyTorch：调用接口.

CUDA：提供接口

Task 1 简单神经网络.

1 可视化原始数据、按训练/验证/测试集划分，说明三者职责.
scatter 散点图 (Data Vistualization.py)

2 搭建 MLP、完成 train/val/test/save/load

1. 生成数据. make_moons(n_samples, noise, random_state).

2. train_test_split(X_temp, y_temp, test_size = __ , random_state, stratify = y_temp).
要划分的 划分给后者的比例 是否按标签原比例划分?

3. 数组 → 张量.

X_train = torch.tensor(X_train, dtype = torch.float32) 32位浮点数.
/.long. 64位有符号整数.

4. 打包成 loader < 标签+数据 > .

train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size, shuffle)
是否按y取出?

5. 定义模型.

```
class MLP(nn.Module):
    def __init__ (self, hidden_units, num_classes):
        super().__init__()
        先调用父类的初始化方法.
        self.net = nn.Sequential(
```

```
            nn.Linear(2, hidden_units)
            nn.ReLU(),
            nn.Linear(hidden_units, 2).

    def forward(self, x):
        return self.net(x).
```

6. 训练

```
torch.manual_seed(SEED)
```
权重初始化、DataLoader 打乱顺序等随机过程

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu').

output_dir = Path(__file__).parent.parent/'outputs'
output_dir.mkdir(parents=True, exist_ok=True).

model = MLP(HIDDEN_UNITS).to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR).

history = {
        'train_loss': [],
            ⋮
        'val_acc': []}

best_val_loss = float('inf')
```
保存历史最低 loss.

```
records = [].
```
创建 records 列表，保存每个epoch.速度/显存等指标.

```
for epoch in range(NUM_EPOCHS):
    if torch.cuda.is_available():
        torch.cuda.reset_peak_memory_stats()
```
重置显存峰值记录，在这个epoch的范围内记录显存峰值。
```
        torch.cuda.synchronize()
```
确保之前的GPU任务全部执行完毕，强制同步测试起点。

```
    t0 = time.perf_counter()
```
记录开始时刻。

```
    model.train()
```
训练模式。
```
    train_loss, train_correct, train_total = 0.0, 0, 0

    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
```
把样本数据移到设备上去
```
        optimizer.zero_grad()
```
清空上一轮累积的梯度。
```
        logits = model(X_batch)
```
得到模型预测分数。
```
        loss = loss_fn(logits, y_batch)
        loss.backward()
```
反向传播。